Loan Processing and Risk Assessment

Presentation & Viva Guide

Everything you need to explain the project from the very beginning: the story, every code file, **exactly where the dataset and the trained model are called**, a slide-by-slide speaking script, a live-demo plan, and model answers to the questions your ma'am is likely to ask.

Course	21CSC305P / Machine Learning
Institution	SRM Institute of Science and Technology, Ramapuram, Chennai
Department	Computer Science and Engineering | Batch 2
Team	Mutheeshwari J (RA2411003020194), Nikitha S (RA2411003020202), Sandra PV (RA2411003020223)
Repository	github.com/Sandraaapv/loanprocessingandriskassesssment
Code reviewed	GitHub commit ab02a56 (11 Oct 2026, 21:52 IST) plus your local fixes described in section 11

How to use this guide

Part A (sections 1-4) builds understanding: the 10 key facts, the story, the folder map and the dataset-versus-model explanation. **Part B (sections 5-7)** walks through the code and the machine learning ideas in plain words. **Part C (sections 8-12)** is for the day itself: speaking script, demo, viva answers, honest limitations and a checklist. Read Part A and section 4 twice.

Contents

1. If you remember only 10 things 3

2. The story: how we built it, step by step 4

3. Project map: every file and why it exists 6

4. The dataset versus the trained model: where each is called 8

5. Code walkthrough, file by file 11

6. Machine learning ideas in simple words 15

7. One prediction, start to finish 18

8. Slide-by-slide speaking script 20

9. Live demo script 23

10. Viva: questions and model answers 24

11. Honest limitations and fixes to make before you present 28

12. Final checklist and cheat sheet 29

1. If you remember only 10 things

These ten facts answer most questions. Learn them as sentences, not as numbers.

1. **The problem.** Deciding a loan by hand is slow and inconsistent between officers. We built a system that predicts approval, gives a risk score and explains the reason instantly.
2. **The dataset.** It is **synthetic** (computer-generated), 5,000 rows, created by `generate_dataset.py` and stored as `loan_dataset.csv`. It has 11 input columns plus the answer column `Loan_Status` (Y or N). 3,773 approved, 1,227 rejected.
3. **Where the dataset is used.** Only at **training time**: `train_model.py` reads it once (line 33). The website never reads it to make a prediction.
4. **Where the trained model is used.** At **run time**: `app.py` loads `loan_rf_model.joblib` when the server starts (lines 51-56) and calls it for every application (line 239). The model is the 'memory' of what it learned from the dataset.
5. **The models.** Two were trained and compared: Logistic Regression (simple baseline) and Random Forest (500 decision trees). Random Forest won on F1-score, so it is the production model.
6. **The results** on 1,000 unseen test rows: accuracy 95.50%, precision 97.84%, recall 96.16%, F1 96.99%, ROC-AUC 98.32%.
7. **The decision is ML + rules.** The model gives a probability. Then bank-style rules apply: poor credit history means reject; EMI above 45% of income (FOIR cap) means reject; above 50% is rejected more strongly.
8. **EMI** is computed with the reducing-balance formula at 8.5% a year. Example: ₹1,50,000 over 360 months = ₹1,153.37 per month.
9. **What matters most.** Credit history (importance 0.5511), then applicant income (0.1825), loan amount (0.0899), co-applicant income (0.0858).
10. **Be honest about limits.** The data is synthetic, so the results prove the method works, not that it would score 95% at a real bank. Say this before anyone asks; it earns trust.

> **One-sentence project summary (say this first)**
> "We built a loan approval and risk assessment system: a Random Forest model trained on a 5,000-record synthetic dataset predicts approval, a bank-style 45% FOIR rule checks affordability, and a FastAPI backend with a web portal shows the decision, risk score and reason instantly."

2. The story: how we built it, step by step

When you present, follow this order. Each step has three parts: **what** we did, **why**, and **where** it lives in the project. This is also the order of the PowerPoint.

Step 1. Choose and understand the problem

What: Banks receive many loan applications. Manual checking depends on the officer, takes time and can be inconsistent. A wrong approval creates a bad loan (NPA); a wrong rejection loses a good customer.

Why: We wanted to predict *approve / reject* and a *risk* level from simple applicant details.

Where: `Slides 4, 8, 9`

Step 2. Read existing work (literature survey)

What: We studied 10 papers and books: Random Forests (Breiman), scikit-learn, credit scoring reviews, benchmarking of classifiers (Lessmann), XGBoost, SMOTE, ROC analysis and so on.

Why: To justify our choices: tree ensembles do very well on credit data, and ROC-AUC and recall are the right metrics.

Where: `Slides 6, 7, 24, 25`

Step 3. Get a dataset

What: Real bank data is confidential, so we **generated a synthetic dataset** with 5,000 applicants. The columns follow the common loan-prediction layout: Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area and the target Loan_Status.

Why: We needed enough rows with realistic patterns (credit history matters, high EMI is risky, higher income is safer).

Where: `generate_dataset.py, loan_dataset.csv, data_dictionary.json | Slide 12`

Step 4. Prepare the data (pre-processing)

What: We fill the few missing values (Self_Employed with 'No', Credit_History with 1, LoanAmount with the median) and convert text into numbers (Male=1, Female=0, Urban=2, Semiurban=1, Rural=0, and so on).

Why: Machine learning models only understand numbers and cannot handle blanks.

Where: `train_model.py lines 41-65 | Slide 12, 13`

Step 5. Split into training and test data

What: 80% (4,000 rows) to learn from, 20% (1,000 rows) kept hidden to test. The split is stratified so both parts keep the same 75% / 25% approved/rejected mix.

Why: Testing on data the model has never seen shows whether it really learned or just memorised.

Where: `train_model.py line 67 | Slide 12`

Step 6. Train and compare models

What: Logistic Regression (a simple straight-line style model, our baseline) and Random Forest (500 decision trees voting). Both are scored on the same 1,000 test rows.

Why: A baseline shows whether the complex model is actually worth it.

Where: `train_model.py lines 75-109 | Slides 14, 17, 18`

Step 7. Evaluate and choose

What: Accuracy, precision, recall, F1 and ROC-AUC, plus the confusion matrix and feature importance. Random Forest had the higher F1 (0.9699 vs 0.9593) and higher recall, so it is chosen automatically.

Why: Recall matters: we do not want to reject many genuinely good applicants.

Where: `train_model.py lines 100-111 | Slides 16-20`

Step 8. Save what was learned

What: The trained model and its helper files are written to the `model/` folder with joblib, together with metrics, feature importance, a manifest (with the dataset's SHA-256 fingerprint) and chart images.

Why: So the website can use the model instantly without training again.

Where: `train_model.py lines 116-153 | Slide 15`

Step 9. Build the backend (FastAPI)

What: `app.py` loads the saved model when it starts and exposes web endpoints: `/predict` (decision), `/model-performance` (metrics), login, signup, applications history and dataset information pages.

Why: A web service lets any front end (or another system) use the model.

Where: `loanapp/backend/app.py | Slides 11, 13, 15`

Step 10. Add banking rules on top of the model

What: From the applicant's numbers we compute the EMI (8.5%, reducing balance), DTI/FOIR and loan-to-income. Rules then override the model when policy is broken (poor credit, EMI above 45% of income).

Why: A pure ML model may approve something a bank policy would never allow. Rules keep it safe and explainable.

Where: `app.py lines 193-322 | Slides 10, 14`

Step 11. Build the web portal

What: One HTML file (`index.html`) with a dashboard, application form, EMI and eligibility planners, application history and a Model & Dataset page. It calls the backend with `fetch()`.

Why: A real user needs a screen, not a terminal.

Where: `loanapp/frontend/index.html, utils/lending.js, utils/formatINR.js | Slides 13, 15`

Step 12. Verify everything is consistent

What: `verify_connection.py` runs 5 checks: the CSV loads, its SHA-256 fingerprint matches the training manifest, the model reproduces the reported accuracy on 200 test rows, the EMI formula is exact, and the stored metrics are correct.

Why: It proves the numbers in the report are not made up.

Where: `verify_connection.py | Slide 15`

Step 13. Deploy and document

What: A `Procfile` and a root `app.py` let the project run on a cloud host such as Render. README files explain how to run and reproduce.

Why: So anyone can reproduce and use the project.

Where: `Procfile, app.py (root), README.md | Slide 23`

3. Project map: every file and why it exists

Folder layout of the repository (the part you must be able to point at):

```
loanprocessingandriskassesssment/
|-- Procfile                  how a cloud host starts the app (uvicorn app:app)
|-- app.py                    tiny file that forwards to loanapp/backend/app.py
|-- requirements.txt          Python libraries needed
|-- README.md
`-- loanapp/
    |-- README.md
    |-- backend/
    |   |-- generate_dataset.py   CREATES the synthetic dataset
    |   |-- loan_dataset.csv      THE DATASET (5,000 rows)
    |   |-- data_dictionary.json  describes every column (units, min, max, median)
    |   |-- train_model.py        READS the dataset, TRAINS models, SAVES the model
    |   |-- model/                everything training produced (see below)
    |   |-- app.py                FastAPI server: LOADS the model, answers /predict
    |   |-- verify_connection.py  5 self-checks
    |   |-- seed_data.py          creates 24 demo applications for the dashboard
    |   |-- data/                 users.json, applications.json (created at run time)
    |   `-- requirements.txt
    `-- frontend/
        |-- index.html            the whole web portal (HTML + CSS + JavaScript)
        `-- utils/
            |-- lending.js        EMI / FOIR / eligibility maths in JavaScript
            `-- formatINR.js      shows money as ₹1,00,000 and ₹40 Lakh
```

What is inside the model/ folder

File	What it is	Who uses it
`loan_rf_model.joblib`	The trained model itself (the Random Forest with 500 trees). The name says rf, but it holds whichever model won.	app.py line 51
`feature_columns.joblib`	The exact list and order of the 11 input columns the model expects.	app.py line 52
`encode_maps.joblib`	The text-to-number dictionaries (Male=1, Urban=2 ...).	app.py line 54
`dependents_map.joblib`	Maps '0','1','2','3+' to 0,1,2,3.	app.py line 55
`scaler.joblib`	StandardScaler fitted on training data. Needed only if Logistic Regression were the best model.	app.py line 53 (used only if LR is best)
`best_model_name.joblib`	The text 'Random Forest'.	app.py line 56
`results_summary.json`	Accuracy, precision, recall, F1, ROC-AUC of both models.	/model-performance, verify_connection
`feature_importance.json`	Importance score of each input.	/model-performance
`training_manifest.json`	When trained, row counts, class balance, seed, SHA-256 of the dataset.	/health, verify_connection
`*.png (8 images)`	Confusion matrix, ROC curve, feature importance, model comparison, each in normal and dark version.	Charts for reports

Why everything is saved in separate files

The model only understands numbers. When a user types 'Urban', the server must convert it to 2 using the **same** dictionary used during training, in the **same** column order. That is why `encode_maps`, `dependents_map` and `feature_columns` are saved next to the model.

4. The dataset versus the trained model: where each is called

The question your ma'am asked last time

She asked: '*Where is the dataset?*' (you showed it) and then '*Where is it called in the code?*' You looked for the dataset and found the **trained model** being called instead. That is **correct behaviour**, and this section explains why.

The idea in plain words

Think of a student. The **dataset** is the textbook. **Training** is studying from the textbook. The **trained model** is what the student now has in their head. During the exam (a user submitting an application) the student does not open the textbook; they answer from what they learned. In the same way, our website never opens `loan_dataset.csv` to decide a loan. It uses the saved model file, which is a compact summary of everything the model learned from the dataset.

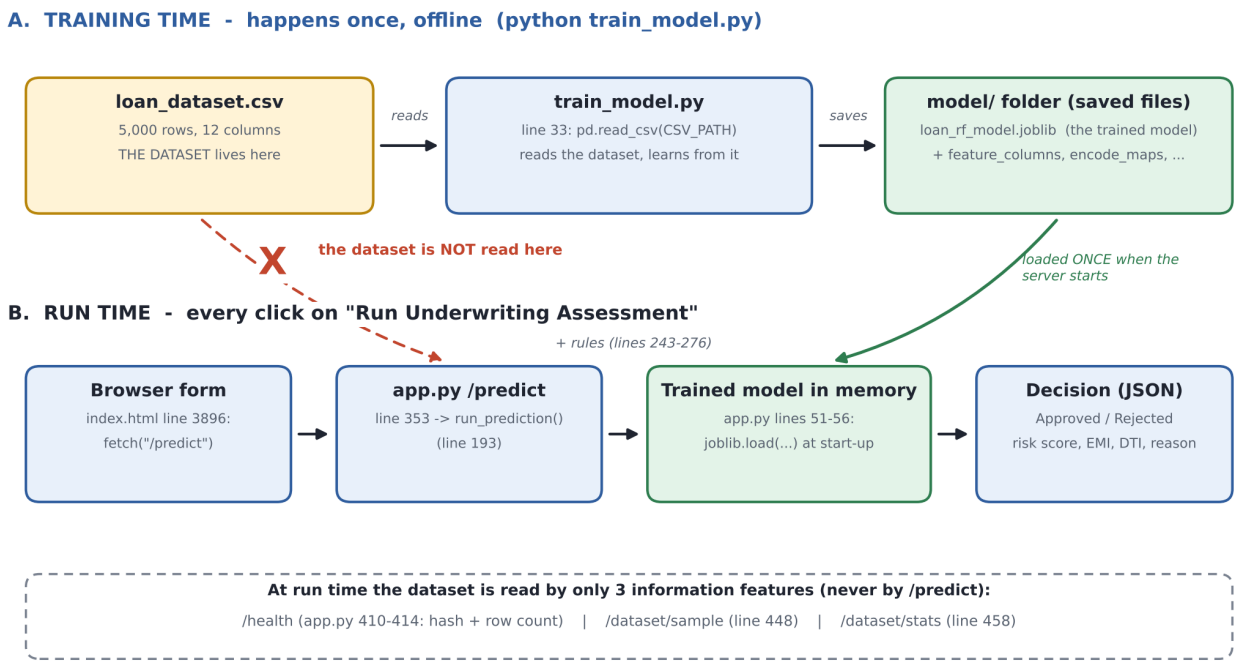


Figure 1. Dataset (training time) versus trained model (run time).

Exact places in the code (line numbers from the GitHub version we reviewed)

If your line numbers differ by a few lines (for example after removing XGBoost), search for the **keyword** in the last column. It never changes.

What happens	File	Line	Search for
Dataset is CREATED and written to disk	`generate_dataset.py`	82	`df.to_csv`
Dataset path is defined	`train_model.py`	28	`CSV_PATH =`
Dataset is READ for training	`train_model.py`	**33**	`pd.read_csv(CSV_PATH)`
Dataset fingerprint (SHA-256) is computed	`train_model.py`	34	`csv_bytes`

What happens	File	Line	Search for
Missing values filled	train_model.py	41-43	fillna
Text converted to numbers (encoding)	train_model.py	45-56	encode_maps
The 11 model input columns chosen	train_model.py	58	feature_cols
Train/test split 80:20, stratified	train_model.py	67	train_test_split(
Models defined (Logistic Regression, Random Forest)	train_model.py	76-87	models = {
Models LEARN from the training data	train_model.py	92 and 96	model.fit(
Models scored on the test data	train_model.py	93-98	model.predict(
Best model picked by F1 (tie: recall)	train_model.py	111	best_name =
Trained model SAVED	train_model.py	**116**	joblib.dump(best_model
Helper files saved (columns, scaler, maps)	train_model.py	117-121	joblib.dump(
Metrics, importance, manifest saved	train_model.py	125, 134, 153	json.dump
Charts saved as PNG	train_model.py	183-239	savefig
Trained model LOADED when server starts	app.py	**51-56**	joblib.load(
Browser sends the application	index.html	3896	fetch("/predict"
/predict endpoint receives it	app.py	353	@app.post("/predict")
Prediction logic starts	app.py	193	def run_prediction
Input row built for the model (11 columns)	app.py	219-232	row = {
Model gives the probability	app.py	**239**	model.predict_proba
Banking rules applied (credit, 45% FOIR, 50% DTI)	app.py	243-276	MAX_FOIR = 45.0
Result saved to applications.json	app.py	356-369	write_json(APPS_FILE

The only places the dataset is used while the website is running

Where	Line	Why it reads the CSV	Used by (index.html)
app.py /health (get_health_status)	410-414	Recomputes the SHA-256 and counts rows to show 'hash matches' (proof the data is unchanged).	fetchHealth, line 4017
app.py /dataset/sample	448	Shows 10 random real rows on the Model & Dataset page.	fetchAndRenderSamples, line 4156
app.py /dataset/stats	458	Computes approval rate by property area, education, credit history, loan size.	fetchDatasetStats, line 4104
verify_connection.py	42	Test 1 and 3: loads the CSV to check schema and re-score 200 test rows.	run manually

The page `/dataset/summary` (line 438) reads `data_dictionary.json`, not the CSV. None of these four places influence a loan decision. They exist so the project is transparent.

What to say if she asks again (say it calmly, point at the lines)

"Ma'am, the dataset is `loan_dataset.csv`. It is read in `train_model.py` at line 33, only during training. After training, we save the learned model with `joblib.dump` at line 116. The web application loads that saved model in `app.py` at line 51 and uses it at line 239 for every prediction. The dataset is not needed at prediction time, just like a student does not need the textbook during the exam. The dataset is read again only by three information pages: health check, dataset sample and dataset statistics."

Show it live in 30 seconds

1) Open `train_model.py` and search `read_csv` → line 33. 2) Search `joblib.dump` → line 116. 3) Open `app.py`, search `joblib.load` → line 51. 4) Search `predict_proba` → line 239. 5) Search `read_csv` in app.py → only lines 448 and 458 (information pages).

5. Code walkthrough, file by file

For each file: **purpose**, **what each block does**, and **what to say**.

5.1 generate_dataset.py : creates the data

Purpose: produce a realistic, repeatable dataset of 5,000 loan applications.

Lines	What the code does	Why
15-16	np.random.seed(42), N = 5000	Seed makes it repeatable: same data every run.
18-23	Random Gender, Married, Dependents, Education, Self_Employed, Property_Area with realistic shares (for example Male 78%, Graduate 78%).	Mimic the mix of a real applicant pool.
25-36	Incomes and loan amounts from a log-normal distribution, clipped (income 1,500-45,000; loan 25-700 thousand).	Real incomes are skewed: many small, few large.
38-41	Loan term (360 most common) and Credit_History (78% good, 22% poor).	Long home-loan terms are common.
43-46	Total income, a rough EMI (0.75% of loan per month) and a DTI proxy.	Used only to create the label.
48-57	A hidden **risk score**: +3.2 for poor credit, + for high DTI, small + for not graduate, self-employed, 3+ dependents, rural; minus for higher income; plus random noise (standard deviation 0.35).	Builds the pattern the model must discover. Noise stops it being a trivial rule.
59-60	Sigmoid turns risk into approval probability; probability above 0.5 means Loan_Status = Y else N.	This is how the answer column is created.
77-80	Blank out 2% of LoanAmount, Credit_History and Self_Employed (100 rows each).	Real data has missing values, so pre-processing has something to fix.
82	Writes loan_dataset.csv.	**Warning:** the path is hard-coded to a folder that does not exist on your laptop (section 11).

Say

"We could not get real bank data because it is confidential, so we generated a synthetic dataset with underwriting logic: poor credit history, high EMI-to-income and low income increase risk, plus random noise. The model then has to *discover* these patterns. We state clearly that this is synthetic."

5.2 train_model.py : reads the dataset, trains, saves

Lines (approx.)	What the code does	Plain meaning
1-26	Imports pandas, numpy, scikit-learn, joblib, matplotlib.	Tools. Optional XGBoost import (lines 21-25) is in the GitHub version; we recommend removing it.
28-34	CSV_PATH points to loan_dataset.csv; **df = pd.read_csv(CSV_PATH)**; SHA-256 of the file is computed.	**The dataset is read here.** The hash is a fingerprint to prove which data trained the model.

Lines (approx.)	What the code does	Plain meaning
36-43	Fill missing: Self_Employed='No', Credit_History=1, LoanAmount=median.	Remove blanks.
45-56	Dependents '3+' becomes 3; Gender, Married, Education, Self_Employed, Property_Area mapped to numbers.	Text to numbers.
58-65	feature_cols = the 11 input columns; X = those columns; y = 1 if Loan_Status is 'Y' else 0.	X = questions, y = answers.
67-70	train_test_split(test_size=0.2, random_state=42, stratify=y).	4,000 to learn, 1,000 to test; same Y/N mix; repeatable.
71-73	StandardScaler fitted on training data only.	Puts numbers on the same scale. Needed for Logistic Regression, not for trees.
75-87	Logistic Regression (max_iter=1000, balanced) and Random Forest (500 trees, max_depth=16, min_samples_split=3, balanced, seed 42).	The two models to compare.
89-109	For each model: **fit** on training data, **predict** on test data, compute accuracy, precision, recall, F1, ROC-AUC.	Learn, then take the exam.
111-113	best_name = model with highest (F1, recall).	Automatic winner: Random Forest.
115-121	**joblib.dump** of the best model, feature columns, scaler, encode maps, dependents map, best name.	**Saves what was learned** into model/.
123-134	Writes results_summary.json and feature_importance.json.	Numbers used by the website and the slides.
136-153	Writes training_manifest.json (row count, class balance, split sizes, seed, time, SHA-256, metrics).	Audit trail.
156-239	Draws and saves 4 charts (confusion matrix, ROC, feature importance, model comparison) in normal and dark colours.	Evidence for the report.

Do not re-run train_model.py right before your presentation

It overwrites the model files, the metrics and the charts and changes the 'trained at' time. Your slides and the verification script both expect today's numbers. Run it only if you intend to update everything.

5.3 app.py (backend) : loads the model, serves the website

Lines	What it does	Plain meaning
1-30	Imports, paths (BASE, MODEL_DIR, CSV_FILE, DATA_DIR), ANNUAL_RATE = 8.5.	Settings. 8.5% is the single interest rate used everywhere.
32-37	Creates data/users.json and applications.json as '[]' if they do not exist.	Storage starts empty if you delete the data files.
39-48	Creates the FastAPI app, enables CORS, serves /static (model charts) and /utils (JavaScript).	Web server set-up.
50-60	**joblib.load** of model, feature_cols, scaler, encode_maps, dependents_map, best_model_name; reads results_summary and feature_importance.	**Trained model enters memory once, at start-up.**
62-74	FEATURE_LABELS: friendly names ('Married_enc' becomes 'Marital Status').	For display.

Lines	What it does	Plain meaning
76-86	read_json, write_json, hash_pw (SHA-256 of password).	Tiny file database and password hashing.
88-127	Pydantic schemas: SignupRequest, LoginRequest, ApplicantData (with defaults), PredictRequest.	Describe and validate the JSON the website sends.
130-176	/signup, /login, /auth/google.	Accounts stored in users.json.
179-191	calculate_reducing_emi(): E = P x r x (1+r)^n / ((1+r)^n - 1).	Monthly instalment.
193-322	**run_prediction()**: EMI, DTI, loan-to-income, build the input row, model.predict_proba, apply rules, top factors, radar scores.	The core of the project (section 7).
324-351	build_explanation(): plain-English reason text.	Why approved / why rejected.
353-376	**/predict**: runs run_prediction, saves the record with a reference code like LG-000148, returns the result.	The endpoint the form calls.
379-394	/applications and /applications/{id}.	History list.
397-403	/model-performance: metrics and feature importance from the JSON files.	Feeds the Model page.
406-436	/health and /api/health: model loaded? row count, SHA-256 match, trained time.	Proof of integrity.
438-515	/dataset/summary, /dataset/sample, /dataset/stats.	Dataset information pages (only place the CSV is read at run time).
517-528	/ returns index.html.	The website is served by the same server.

5.4 Frontend: index.html, lending.js, formatINR.js

- **index.html** (about 4,250 lines) is the whole portal: HTML structure, CSS styling and JavaScript logic in one file. Fonts come from Google Fonts. It uses **plain JavaScript**, not React.
- **Navigation**: `switchTab(tabKey)` (line 3037) shows one panel and hides the others. Panels: Overview, Updates, Start Application, Applications, Offers, Compare, EMI Planner, Eligibility Meter, Payoff Planner, Vault, EMI Calendar, Credit Pulse, Model & Dataset, Help, Settings.
- **Start Application** form (line ~2047) holds the 11 inputs. `handleApplySubmit()` (line 3867) reads them, converts the loan amount from rupees to thousands (line 3893), and does `fetch("/predict")` (line 3896). It shows status, probability, risk score, EMI, DTI, loan-to-income and the explanation.
- **Model & Dataset** panel (line 2631) calls `fetchModelPerformance()` (4056), `fetchHealth()` (4017), `fetchDatasetSummary()` (4077), `fetchDatasetStats()` (4104) and `fetchAndRenderSamples()` (4156).
- **Login/Signup**: handleLoginSubmit (3190) and handleSignupSubmit (3229) call /login and /signup; the user is remembered in the browser's localStorage.
- **lending.js** (imported at line 2932): emi(), foir(), maxEligibleLoan() and assess(), with MAX_FOIR = 45. These power the dashboard cards, EMI Planner, Eligibility Meter and Payoff Planner. It is **not** the ML model.
- **formatINR.js**: shows ₹1,00,000 style numbers and compact forms such as ₹40 Lakh and ₹2.5 Cr.

Two engines, do not confuse them

The **ML + rules engine** lives in Python (`app.py`) and decides Approved or Rejected. The **lending engine** in `lending.js` does planner maths (EMI, headroom) in the browser. Both use the same 45% FOIR limit and the same 8.5% rate, so numbers agree.

5.5 verify_connection.py : the 5 self-checks

Test	What it checks	Why it matters
1	CSV exists, loads (5,000 rows), every column is in data_dictionary.json.	Data and its description agree.
2	SHA-256 of the CSV equals the one stored in training_manifest.json.	The dataset has not changed since the model was trained.
3	Re-scores 200 random rows from the **test split** (same seed 42) and compares accuracy and recall with the saved values, allowing 1.5 percentage points.	The saved model really performs as reported. Your run: 94.50% accuracy, 95.77% recall.
4	EMI for ₹1,50,000 at 8.5% for 360 months must equal ₹1,153.37.	The finance maths is correct.
5	Stored metrics equal 0.955, 0.9784, 0.9616, 0.9699, 0.9832.	Numbers on slides equal numbers on disk.

Your run printed **ALL 5 PIPELINE VERIFICATION TESTS PASSED**. Show this output in the demo; it is strong evidence. Note: Test 5 compares against fixed numbers, so if you ever retrain and the numbers move, it will fail until you update it.

5.6 seed_data.py : demo data

Builds 24 hand-written applicant profiles (strong, moderate, high-risk), runs each through `run_prediction()` and writes them to `data/applications.json` with dates between late September and 8 October 2026. This fills the dashboard history so it does not look empty. These are demo records, not training data.

6. Machine learning ideas in simple words

Use these explanations if she asks 'what is ...?'. Each has a real-life comparison and, where useful, our own number.

Features and target

Features are the questions we ask (income, loan amount, credit history ...). The target is the answer we want to predict (Loan_Status: Y or N). We have 11 features. The model learns the link between them.

Training data and test data

Like a class: practice questions (4,000 rows) versus a surprise exam (1,000 rows). We report only the exam score, because a student who memorised answers would score full marks on practice questions.

stratify=y

When splitting, keep the same share of approved/rejected in both parts (about 75% / 25%). Otherwise the test set could be accidentally easier.

random_state=42 (seed)

Makes the 'random' split and random trees repeatable. Anyone running the code gets the same numbers.

Encoding

Turning words into numbers: Male=1, Female=0; Urban=2, Semiurban=1, Rural=0. Models cannot read words.

Missing values

Blanks are filled so the model can use every row: Credit_History blank means 1 (good), Self_Employed blank means 'No', LoanAmount blank means the median.

Scaling (StandardScaler)

Brings columns like income (thousands) and credit history (0 or 1) to a similar size. Logistic Regression needs it; decision trees do not, because trees only ask 'is income above X?'.

Logistic Regression (baseline)

A simple model that draws one smooth boundary between approve and reject. It is fast and easy to explain, so it is a good yardstick. Ours scored 94.00% accuracy.

Decision tree

A flowchart of yes/no questions: 'credit history good? income above 3,000? loan below 150?' that ends in approve or reject.

Random Forest

Many decision trees (we use 500), each trained on a random part of the data and random features. They vote. One tree can be wrong; 500 voting trees are far more stable. It also tells us which features mattered. This is an *ensemble*.

class_weight='balanced'

Only 25% of rows are rejections. This setting makes mistakes on the rare class count more, so the model does not simply say 'approve' to everyone.

Hyperparameters we set

n_estimators=500 (trees), max_depth=16 (how deep each tree can grow, limits over-fitting), min_samples_split=3, min_samples_leaf=1.

predict versus predict_proba

predict gives the final class (0 or 1). predict_proba gives the probability, such as 0.9885 = 98.85% chance of approval. We use the probability, because we then apply rules and show a risk score.

Over-fitting

Memorising training data and failing on new data. We check by scoring on unseen test rows (95.50%). We have not done cross-validation; that is listed as future work.

The metrics, with our own confusion matrix

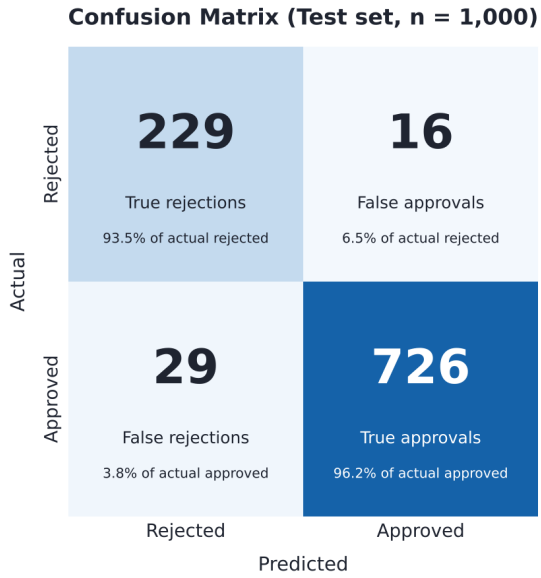


Figure 2. Test set of 1,000 rows. TN = 229, FP = 16, FN = 29, TP = 726.

Metric	Meaning in plain words	Calculation with our numbers	Result
Accuracy	Out of all 1,000 decisions, how many were right.	(726 + 229) / 1,000	95.50%
Precision	When the model says 'approve', how often is it right.	726 / (726 + 16) = 726 / 742	97.84%
Recall (sensitivity)	Of all truly good applicants, how many did we approve.	726 / (726 + 29) = 726 / 755	96.16%
F1-score	One number balancing precision and recall.	2 x P x R / (P + R)	96.99%
Specificity	Of all truly bad applicants, how many did we reject.	229 / (229 + 16) = 229 / 245	93.47%
ROC-AUC	Chance that a random good applicant gets a higher score than a random bad one. 100% is perfect, 50% is guessing.	Area under the ROC curve (from probabilities)	98.32%

False approval (16) means a bad loan is given: costly for the bank. **False rejection (29)** means a good customer is lost. In our test the model makes more false rejections than false approvals.

Feature importance

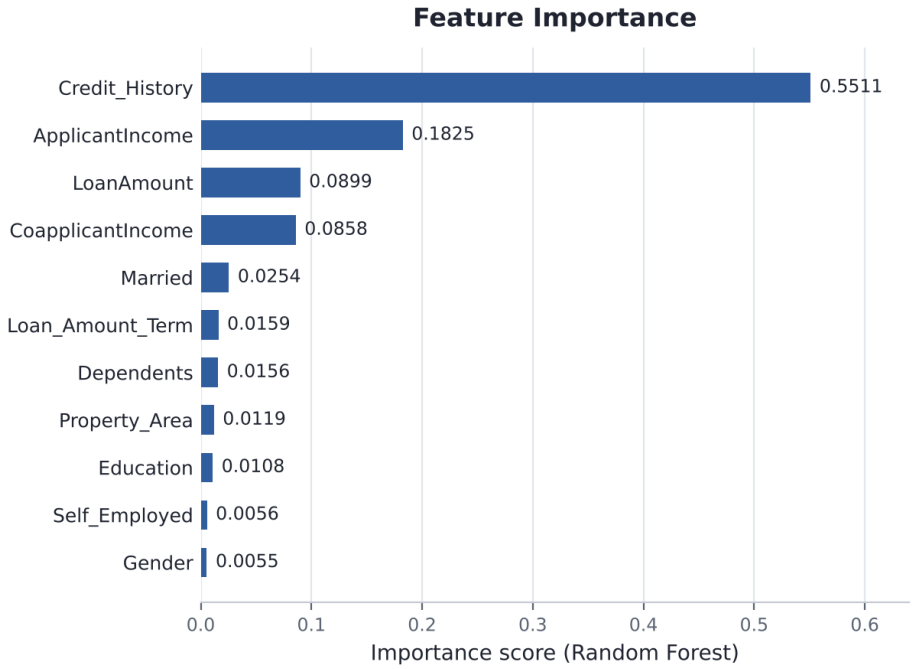


Figure 3. Importance of each input in the Random Forest (scores add up to 1).

Computed by the forest itself (`feature_importances_`): how much each input reduces uncertainty across all trees. Credit history alone is 55%, because in the data generator it is the biggest risk driver. **DTI, EMI and loan-to-income are not on this chart** because they are not model inputs; they are computed in the rule layer.

7. One prediction, start to finish

This is the path of one click on **Run Underwriting Assessment**. Follow it in this order when you demo and explain.

1. **Form (index.html).** The user enters 11 inputs. The loan amount is in rupees; line 3893 divides by 1,000 because the model learned loan amounts in thousands.
2. **fetch("/predict") (line 3896)** sends a JSON message to the server.
3. **/predict (app.py line 353)** calls `run_prediction()`.
4. **EMI (line 208)** from principal = loan x 1,000, rate 8.5%, term in months.
5. **DTI (line 212)** = EMI / total monthly income x 100. **Loan-to-income** = loan / (total income x 12).
6. **Input row (lines 219-232)**: the 11 columns encoded as numbers, in the saved column order.
7. **ML probability (line 239)**: `model.predict_proba(X)` returns p, the chance of approval.
8. **Rules (lines 243-276)** adjust or override p, in the fixed order shown in Figure 4.
9. **Risk score** = (1 - final probability) x 100. A readable explanation is built (line 324), and the top 5 factors and a 5-point radar are added.
10. **Saved and returned**: the application is written to applications.json with a reference like LG-000148, and the result goes back to the browser, which paints the result card.

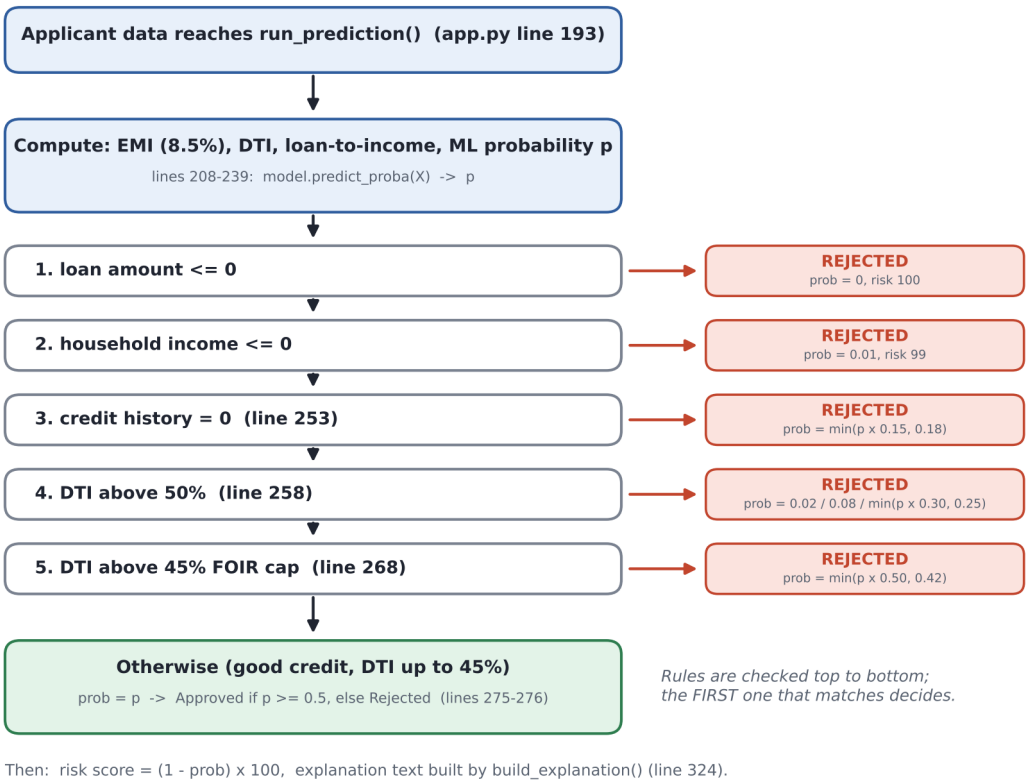


Figure 4. Rule order inside run_prediction(). The first rule that matches decides.

Four real examples (run on the actual project code)

Use these in the live demo. Enter amounts in the form as shown (rupees, not thousands). Income values are the monthly rupee amounts from the form; all other fields are the defaults: Male, Married, 0 dependents, Graduate, Not self-employed unless stated.

Case	Inputs	EMI / DTI	Result
A Good applicant (form defaults)	Income 4,500 + 1,500; loan ₹1,50,000; 360 months; credit history 1; Urban	₹1,153.37 / 19.2%	**Approved** probability 0.9885, risk 1.1
B Same, poor credit	Same as A but credit history = 0	₹1,153.37 / 19.2%	**Rejected** probability 0.003, risk 99.7 (rule 3: credit history)
C Big, short loan	Income 4,500 + 1,500; loan ₹4,00,000; 120 months; credit history 1	₹4,959.43 / 82.7%	**Rejected** probability 0.08, risk 92.0 (rule 4: DTI above 50%)
D Low income, rural	Female, single, 2 dependents, not graduate, self-employed; income 2,000; loan ₹1,00,000; 60 months; Rural	₹2,051.65 / 102.6%	**Rejected** probability 0.02, risk 98.0 (DTI above 100%)

Hand-check of case A: r = 8.5 / 12 / 100 = 0.0070833, n = 360. EMI = 150,000 x r x $(1+r)^n$ / ($(1+r)^n$ - 1) = ₹1,153.37. DTI = 1,153.37 / 6,000 = 19.2%. Loan-to-income = 150,000 / (6,000 x 12) = 2.08. Case B has the same EMI and DTI; the credit-history rule alone flips the decision. This shows the rules working and is a good moment to explain the hybrid design.

About the 'top factors' shown in the result

The factor list on the result card shows the model's **overall** importances (the same for every applicant), with a positive/negative label chosen from simple conditions (for example credit history 0 is negative). It is not a per-person SHAP explanation. Say 'key factors' not 'exact reasons for this person'.

8. Slide-by-slide speaking script

Target length is 12-15 minutes. A suggested split between the three of you is below; change it as you like. Say the *italic* text in your own words. 'If asked' lines are quick safety nets.

Speaker (suggestion)	Slides	Time
Mutheeshwari	1-8: title, agenda, abstract, scope, introduction, literature survey, problem	about 4 min
Nikitha	9-15: objective, proposed work, architecture, dataset, modules, algorithm, implementation + live demo	about 6 min
Sandra	16-25: results, comparison, conclusion, future work, references	about 4 min

Slide 1: Title

“Good morning ma'am. We are Batch 2: Mutheeshwari, Nikitha and Sandra. Our project is Loan Processing and Risk Assessment using machine learning, for 21CSC305P.”

Slide 2: Agenda

“We will go from the problem, to the data, the model, the system, the results and finally the limitations and future work. We will also show a live demo.”

Slide 3: Abstract

“Loan approval is risk-sensitive and slow when manual. We built an automated system. We generated a synthetic dataset of 5,000 applications and compared Logistic Regression with Random Forest. The 500-tree Random Forest is deployed and reaches 95.50% accuracy, 96.16% recall and 98.32% ROC-AUC on 1,000 unseen records. It is combined with a 45% FOIR rule and served through FastAPI and a web portal.”

Tip: If asked 'why synthetic': real bank data is confidential.

Slide 4: Scope and Motivation

“A wrong approval creates a non-performing asset; a wrong rejection loses a customer. Our scope is eligibility prediction, risk assessment and a web portal. It supports decisions, it does not replace officers. And because the data is synthetic, it must be validated on real data before real use.”

Slide 5: Introduction

“Machine learning learns from past applications. We trained two models, Logistic Regression as a baseline and Random Forest, and deployed the better one. It is a full-stack application: FastAPI backend and a single-page web portal.”

Tip: If asked about the earlier prototype: it was a single-script Streamlit version (as stated on the slide).

Slide 6: Literature survey (1/2)

“We studied ten works. Breiman's Random Forests reduce variance; scikit-learn is our toolkit; Hand and Henley and Khandani et al. show statistical and machine learning methods in credit scoring; Lessmann et al. found ensembles among the best classifiers.”

Tip: Do not read the table. Pick two papers.

Slide 7: Literature survey (2/2)

“XGBoost shows boosted trees are strong on tabular data; SMOTE handles imbalance, which we mention as future work; Fawcett explains ROC-AUC, our key metric; Thomas motivates retraining; Hastie et al. cover cross-validation and bias-variance.”

Slide 8: Problem statement

“Manual review is time-consuming, depends on the officer and can be inconsistent. Banks need an automated, data-driven system that predicts approval, quantifies risk, responds in real time and stays transparent.”

Slide 9: Objective

“Eight objectives: automate eligibility prediction; compute EMI, DTI and loan-to-income and apply the 45% rule; compare Logistic Regression and Random Forest; evaluate with accuracy, precision, recall, F1, ROC-AUC, confusion matrix and feature importance; serve it through FastAPI; and provide a web portal.”

Slide 10: Proposed work

“Applicant details go in, the data is cleaned and encoded into 11 features, two models are trained and the best is saved. At prediction time the backend also computes EMI at 8.5% and DTI and applies the 45% FOIR rule. The final decision is the model's probability combined with underwriting rules, with a risk score and an explanation.”

Tip: Key sentence: ML + rules.

Slide 11: Architecture

“Top row is the online path: the applicant uses the browser, the lendr portal sends the request to the FastAPI backend, which uses the Random Forest and the FOIR rule layer, and stores records in JSON files. Bottom row is offline: the dataset goes into train_model.py, which saves the model files that the backend loads when it starts.”

Tip: Point at the arrow from 'Saved artifacts' to the backend: this is where the trained model is called, not the dataset.

Slide 12: Dataset and features

“5,000 synthetic records, 3,773 approved and 1,227 rejected. 4,000 for training, 1,000 for testing, stratified. Eleven input features. Missing values are filled and categories encoded. EMI, DTI and loan-to-income are derived at prediction time for the rule layer. The limitation is that the data is synthetic.”

Tip: Be ready to open generate_dataset.py and loan_dataset.csv.

Slide 13: Modules

“Six modules: pre-processing, underwriting engine, model training, evaluation, FastAPI backend and the web portal frontend.”

Tip: Match each row to a file from section 3 of this guide.

Slide 14: Algorithm

“Logistic Regression is the baseline. Random Forest builds 500 trees on random samples; their vote reduces variance, and it is class-balanced. The rule layer rejects poor credit history or DTI above 50%, and sharply reduces the probability above the 45% cap. Feature importance shows what drives decisions.”

Tip: If asked 'is it an ensemble': yes, a Random Forest is an ensemble of trees.

Slide 15: Implementation

“generate_dataset.py creates the data. train_model.py pre-processes, trains, compares and saves. The FastAPI server on port 8000 loads the saved model and serves /predict and /model-performance, and also serves the portal. verify_connection.py runs five automated checks.”

Tip: Then go to the demo (section 9).

Slide 16: Results and discussion

“On 1,000 unseen records: accuracy 95.50%, precision 97.84%, recall 96.16%, F1 96.99%, ROC-AUC 98.32%. Recall means only 29 of 755 good applicants were rejected. Remember the data is synthetic.”

Slide 17: Model benchmark

“Random Forest beats Logistic Regression on accuracy, recall, F1 and ROC-AUC. Logistic Regression has slightly higher precision.”

Slide 18: Baseline vs Random Forest

“Gains over the baseline: +1.50 accuracy, +2.52 recall, +1.06 F1, +0.18 ROC-AUC; precision is 0.49 lower. We choose Random Forest because it misses fewer creditworthy applicants.”

Slide 19: Confusion matrix

“726 true approvals, 229 true rejections, 16 false approvals and 29 false rejections. Here false rejections are the larger error. Specificity is 93.47%.”

Tip: Know TP/TN/FP/FN meanings.

Slide 20: Feature importance

“Credit history is by far the strongest at 0.5511, then applicant income, loan amount and co-applicant income. Credit history plus applicant income is about 73% of total importance. Gender and self-employment are below 0.006.”

Tip: If asked 'why not DTI': it is not a model input; it is used in the rule layer.

Slide 21: Comparison with existing work

“Compared with manual review and the earlier single-script prototype, ours is automated, measurable, consistent, split into backend and frontend, and exposes a REST API.”

Slide 22: Conclusion

“We built and verified an automated loan approval and risk assessment system. Random Forest reached 95.50% accuracy and 96.16% recall. Credit history and income matter most. It is a decision-support tool, and it needs validation on real data.”

Slide 23: Future work

“Real data and cross-validation, SHAP explanations and a fairness audit, threshold tuning, a real database with stronger password hashing, Docker and cloud deployment with drift monitoring, and integration with document verification and credit bureaus.”

Slide 24: References (papers)

“Ten references; we used them to justify the algorithm and the metrics.”

Tip: Do not read them out.

Slide 25: References (resources)

“scikit-learn, Kaggle, IBM, RBI, Python and Google ML documentation. Thank you, we are happy to take questions.”

Tip: Hand over to the demo or Q&A.;

9. Live demo script

Always demo with the server running

If the backend is not reachable, the form silently falls back to a simple local rule (index.html lines ~3975-4012) and shows **placeholder** numbers (92.0% or 12.0%). That would look wrong to anyone who knows the code. Start the server before you begin and keep the terminal visible.

Before the session (5 minutes, the day before and the morning of)

```
git pull                                  # make sure you have the latest fixes
cd loanapp/backend
pip install -r requirements.txt
python verify_connection.py               # must say: ALL 5 ... TESTS PASSED
python -m uvicorn app:app --host 127.0.0.1 --port 8000
# open  http://localhost:8000   in the browser
```

Demo steps (about 4 minutes)

1. **Show the terminal** running `uvicorn`. Say: 'The backend is FastAPI; it already loaded the trained model at start-up.'
2. **Open http://localhost:8000**, sign up or log in (the account is saved in users.json).
3. Click **+ Start application** (left side). Keep the defaults (income 4,500 + 1,500; loan ₹1,30,000 or type 150000; 360 months; credit history 1). Click **Run Underwriting Assessment** → **Approved**. Point at EMI, DTI, probability and the explanation.
4. Change **Credit Bureau Guideline to 0** → run again → **Rejected**. Say: 'Same income and loan; only credit history changed. The model and the rule both treat it as the biggest risk.'
5. Set **loan ₹4,00,000 and 120 months** with credit history 1 → **Rejected**, DTI about 82.7%. Say: 'The EMI is 82.7% of income, far above our 45% FOIR cap.'
6. Open **Model & Dataset** (in the left menu). Show accuracy, precision, recall, F1, ROC-AUC, the dataset statistics and the live sample rows. Say: 'These numbers come from /model-performance and /dataset/stats.'
7. Open **http://localhost:8000/health** → show `hash_matches_dataset: true`. Say: 'This proves the dataset is the one the model trained on.'
8. Open **http://localhost:8000/docs** → expand POST /predict. Say: 'FastAPI generates this documentation automatically.'
9. In the terminal run `python verify_connection.py` → all 5 tests pass.

Backup plan

Take screenshots of: the approved result, the rejected result, the Model page, /health and the verification output. Keep them in a folder or at the end of the slide deck. If the laptop or network misbehaves, switch to screenshots and keep talking.

10. Viva: questions and model answers

Answers are written to be spoken. Where a line number helps, give it. If you do not know something, say 'We did not implement that; it is in our future work' - never guess.

A. Dataset and data

Q. Where is the dataset?

In `loanapp/backend/loan_dataset.csv`: 5,000 rows, 12 columns. Its columns, units and ranges are described in `data_dictionary.json`.

Q. Where is the dataset called in the code?

Only in `train_model.py` line 33 (`pd.read_csv`) for training, and in three information endpoints of `app.py`: /health (410-414), /dataset/sample (448), /dataset/stats (458). Never in /predict.

Q. Then what is called when a user applies?

The trained model, saved as loan_rf_model.joblib. app.py loads it at lines 51-56 and calls `predict_proba` at line 239.

Q. Why not use the dataset directly for prediction?

The model is a compact summary of what was learned. Reading 5,000 rows for every request would be slow and would not produce a prediction anyway. Training is done once, prediction many times.

Q. Is the dataset real?

No. It is synthetic, generated by `generate_dataset.py` because real bank data is confidential. We state this on slides 4, 12, 16 and 22.

Q. How was the Loan_Status label created?

From a hidden risk score: poor credit history adds a lot of risk, high EMI-to-income adds risk, higher income reduces it, plus small effects of education, employment, dependents and area, plus random noise. A sigmoid converts the risk to an approval probability; above 0.5 means Y (lines 48-60).

Q. How did you handle missing values?

The generator blanks 2% of three columns. In training we fill Self_Employed with 'No', Credit_History with 1 and LoanAmount with the median (lines 41-43).

Q. Is the data balanced?

No: 75.46% approved, 24.54% rejected. We use class_weight='balanced' for both models and stratified splitting.

Q. Can you regenerate the dataset?

Yes, with the seed 42 the generator is repeatable. We tested that regenerating gives a file with the identical SHA-256. But the output path inside the script is hard-coded and must be changed first (section 11).

B. Model and machine learning

Q. Why Random Forest?

It handles mixed numeric and categorical data, captures non-linear rules, resists over-fitting through averaging, needs no scaling and gives feature importance. It also beat Logistic Regression on F1 (0.9699 vs 0.9593).

Q. Why is accuracy so high? Is it over-fitting or leakage?

The test rows were never used for training, so it is not memorisation. But the label came from a formula on the same features plus noise, so a flexible model can recover it. Real bank data is noisier and we would expect lower accuracy. That is why we call the result a demonstration of the method.

Q. Why 80:20 split? What is stratify?

80:20 is a common choice: enough to learn, enough to test. Stratify keeps the approved/rejected ratio the same in both parts.

Q. What is random_state=42?

A fixed seed so the split and the random trees are repeatable.

Q. Why do you scale for Logistic Regression but not for Random Forest?

Logistic Regression is sensitive to the size of numbers; trees only compare values to thresholds, so scaling does not change them (lines 71-98).

Q. Why is scaler.joblib saved if Random Forest is used?

It is saved in case Logistic Regression is the best model. app.py applies it only if the best model name is 'Logistic Regression'. With Random Forest it is not used.

Q. Is it an ensemble model?

A Random Forest is itself an ensemble of 500 trees. We do not combine different model types.

Q. Explain precision and recall for this project.

Precision 97.84%: of the applications we approve, 97.84% are truly good. Recall 96.16%: of the truly good applicants, we approve 96.16%.

Q. Which error is worse?

False approvals (16) give a loan that may default, which is costly to the bank. False rejections (29) lose good customers. The 45% FOIR and credit rules further reduce risky approvals.

Q. What is ROC-AUC?

The probability that a random good applicant receives a higher score than a random bad applicant. Ours is 98.32%; 50% would be guessing.

Q. How is feature importance calculated?

The Random Forest measures how much each feature reduces impurity across all trees; the scores add up to 1. Credit history is 0.5511.

Q. Did you use cross-validation or tuning?

No. We used a single stratified hold-out split and hand-chosen settings (500 trees, depth 16). Cross-validation is in the future work.

Q. Why is there XGBoost in the code?

An optional import in the GitHub version that was not used in the final results; the reported results are Logistic Regression and Random Forest only. (We recommend removing it, section 11.)

C. System and code

Q. Why rules on top of the model?

A model trained on synthetic data can approve something a bank's policy forbids. Rules guarantee policy (credit history, 45% FOIR cap), make decisions explainable and let us show a clear reason.

Q. What is FOIR / DTI and why 45%?

FOIR is the fixed obligations to income ratio: monthly EMI divided by monthly income. We use 45% as a prudent limit in our project policy (the code constant is MAX_FOIR at app.py line 243 and lending.js line 8). Above 50% we reject more strongly.

Q. How is EMI calculated?

Reducing-balance formula E = P r (1+r)^n / ((1+r)^n - 1), r = annual rate / 12 / 100. At 8.5%, ₹1,50,000 over 360 months is ₹1,153.37 (app.py line 179; verified in test 4).

Q. Why is the interest rate fixed at 8.5%?

It is a single benchmark constant (ANNUAL_RATE, app.py line 30). Real banks price by profile; that is future work.

Q. What does /predict return?

Status, probability, risk score, top factors, estimated EMI, DTI percent, loan-to-income ratio, a radar of five scores and a text explanation, plus a reference code and time.

Q. How are the frontend and backend connected?

FastAPI serves index.html at '/', so the page and the API share one address and the page simply calls fetch("/predict").

Q. What are FastAPI and uvicorn?

FastAPI is the Python web framework that defines the endpoints; uvicorn is the server that runs it. FastAPI also gives automatic docs at /docs.

Q. What is joblib?

A library that saves Python objects, here the trained model, to a file and loads them back.

Q. What is the SHA-256 for?

A fingerprint of the dataset file. If one byte changes, the hash changes. Test 2 and /health compare it with the value recorded at training.

Q. How are passwords stored?

As an unsalted SHA-256 hash (app.py line 85). That is acceptable for a demo but not for production, where bcrypt or argon2 would be used.

Q. Where is user and application data stored?

In two JSON files under loanapp/backend/data. A real system would use a database (future work).

Q. How can it be deployed?

The Procfile (uvicorn app:app) and the root app.py are ready for a host such as Render.

Q. What if the user enters the loan amount in thousands by mistake?

The form shows a warning when the value looks like thousands or is outside the normal range (index.html lines 3763-3793).

D. Limitations and ethics

Q. What are the limitations?

Synthetic data; hold-out validation only; fixed interest rate; JSON storage and simple password hashing; global (not per-applicant) explanations; no document or bureau verification.

Q. Is using Gender in the model fair?

Gender is an input in our dataset format but its importance is tiny (0.0055). Many lenders may not use it. A fairness audit and removing sensitive attributes are in our future work.

Q. Would this work at a real bank?

Not as is. It needs real, validated data, a fairness and compliance review, cross-validation, monitoring for drift and officer oversight. It is a decision-support prototype.

Q. What would you improve first?

Train and validate on real data with cross-validation, add SHAP explanations and a fairness check, then replace JSON storage with a database.

11. Honest limitations and fixes to make before you present

These are real findings from reading the code. Fix the first four; know the rest in case she asks.

#	Finding	What to do
1	**generate_dataset.py line 82** writes to `/home/claude/loanapp/backend/loan_dataset.csv`, a path that does not exist on your laptop. If anyone runs the script it will fail or write in the wrong place.	Change to `df.to_csv(Path(__file__).parent / 'loan_dataset.csv', index=False)` (add `from pathlib import Path`). We tested the script with the path fixed: it regenerates a file with the **identical SHA-256**, so the data is fully reproducible. Still, do not run it before the presentation.
2	On the GitHub copy we reviewed (latest commit 21:52 IST, 11 Oct) the README still said **200 trees** and **cross-validated**, XGBoost was still in train_model.py and requirements, and users.json / applications.json were still committed.	Your local fixes (500 trees, no cross-validation claim, XGBoost removed, data files untracked) must be **committed and pushed**. Afterwards open the repository page and check README.md, requirements.txt and that the data folder is gone.
3	After removing the data files from Git, old versions remain in history.	If the accounts are real, purge history with `git filter-repo` (see the earlier Antigravity prompt) or accept and change those passwords.
4	**Fallback numbers**: if the backend is down, the form shows invented values (92.0% / 12.0%).	Always demo with the server running (section 9).
5	**scaler.joblib** is saved but unused (Random Forest does not need it).	Know the answer in section 10-B. No change needed.
6	**'Top factors'** are global importances, not a per-person explanation.	Call them 'key factors'. SHAP is future work.
7	**Gender** is a model input (importance 0.0055).	Mention the fairness audit in future work.
8	**Passwords**: unsalted SHA-256; Google sign-in endpoint trusts the email it is sent; CORS allows every origin; data in JSON files.	Fine for a demo. Say 'prototype' and list as future work.
9	**verify_connection.py test 5** checks fixed numbers.	If you retrain and numbers change, update the test or it will fail.
10	Slide 5 and slide 21 mention an **earlier Streamlit prototype**; no Streamlit file is in the repository.	Be ready to say it was our first version and was replaced; or remove the mention if it is not accurate.

Slide and code must say the same thing

After the fixes, the three sources must agree: **README** (500 trees, hold-out split, 5,000 rows, Random Forest + Logistic Regression), **code** (train_model.py), and **slides** (the updated text and the three new images for slides 11, 19 and 20).

12. Final checklist and cheat sheet

The day before

- Fixes from section 11 are pushed to GitHub; repository page shows the new README.
- Slides updated with the new text and the three new images (slides 11, 19, 20); slide 1 supervisor name filled in.
- `python verify_connection.py` prints ALL 5 TESTS PASSED.
- Server starts and the demo cases A, B, C work. Screenshots saved as backup.
- Each person has read their slides' script and the viva answers in section 10.
- Laptop charged; project folder and slides copied to a pen drive and cloud storage.

Ten minutes before

- Start the server and open http://localhost:8000; log in once so the page is ready.
- Open in separate tabs/windows: train_model.py, app.py, generate_dataset.py, loan_dataset.csv, /health.
- Do **not** run train_model.py or generate_dataset.py.

Key numbers on one page

Item	Value
Dataset	5,000 synthetic rows; 12 columns (11 inputs + Loan_Status); 3,773 Y / 1,227 N (75.46% / 24.54%)
Split	4,000 train / 1,000 test; test = 755 approved + 245 rejected; seed 42; stratified
Random Forest	500 trees, max_depth 16, min_samples_split 3, min_samples_leaf 1, class_weight balanced
RF results	Accuracy 95.50%, Precision 97.84%, Recall 96.16%, F1 96.99%, ROC-AUC 98.32%
Logistic Regression	Accuracy 94.00%, Precision 98.33%, Recall 93.64%, F1 95.93%, ROC-AUC 98.14%
Confusion matrix	TN 229, FP 16, FN 29, TP 726
Top importances	Credit_History 0.5511, ApplicantIncome 0.1825, LoanAmount 0.0899, CoapplicantIncome 0.0858
Rules	Credit history 0 → reject; DTI above 50% → reject; DTI above 45% → reject; else approve if probability ≥ 0.5
Finance constants	Interest 8.5% a year, reducing balance; FOIR cap 45%; ₹1,50,000 over 360 months = ₹1,153.37
Dataset SHA-256	bf0cdb8d524888975f16cf42a5881f5ac311df11f561cb03ee060df81b2c3678
Run command	cd loanapp/backend; python -m uvicorn app:app --host 127.0.0.1 --port 8000; open http://localhost:8000

Glossary

Term	Meaning
EMI	Equated Monthly Instalment: the fixed amount paid every month.
DTI / FOIR	Share of monthly income that goes to loan payments. We cap it at 45%.
NPA	Non-performing asset: a loan that is not being repaid.

Term	Meaning
Synthetic data	Data generated by a program, not collected from real people.
Pipeline	The chain: data → clean → train → evaluate → save → serve.
Endpoint	A web address the server answers, for example /predict.
joblib	Saves and loads trained models.
Hash (SHA-256)	A fingerprint of a file that changes if the file changes.
Ensemble	Many models voting together (a Random Forest is an ensemble of trees).